Phase 1: Binary Heap Foundations

1. Current Progress

This is the first phase of the heap curriculum. At this point, you are starting from the basics.

Before learning insertion, deletion, heapify, heap sort, or priority queues, you must first understand what a heap is and how it is stored.

In this phase, you will learn:

- What a heap is
- What a complete binary tree is
- Why heaps are stored in arrays
- Why this curriculum ignores index `0`
- Parent and child index formulas
- Max heap
- Min heap
- Duplicates in heaps
- Why heap height is `O(log N)`
- Why heaps are not good for searching
- Why gaps are not allowed
- A small C program to understand heap array relationships

---

2. What Is a Heap?

A **heap** is a special type of **binary tree**.

A binary tree is a tree where each node can have at most two children:

- Left child
- Right child

But not every binary tree is a heap.

To be a heap, the tree must follow two important rules:

1. **Shape rule**
2. **Ordering rule**

Simple meaning:

> A heap is a binary tree that has a fixed shape and a fixed parent-child order.

---

3. The Two Rules of a Heap

3.1 Rule 1: Complete Binary Tree Shape

A heap must be a **complete binary tree**.

A complete binary tree means:

- Every level is completely filled, except possibly the last level.
- The last level must be filled from **left to right**.
- There must be no holes or missing spaces in the middle.

Complete Binary Tree Example

```
        40
      /
    35        30
   /   \     /
 15    10  25    5
```

This tree is complete because the nodes are filled level by level, from left to right.

Incomplete Binary Tree Example

```
        40
      /
    35        30
      \     /
      10  25    5
```

This is not complete because the left child of `35` is missing, but later nodes still exist.

That creates a gap.

A heap does not allow this.

3.2 Rule 2: Heap Ordering Rule

After the shape rule, the heap must also follow an ordering rule.

There are two common types of heaps:

1. **Max heap**
2. **Min heap**

Max Heap Rule

In a max heap:

```
Parent >= Child
```

This means every parent must be greater than or equal to its children.

So the largest value is always at the root.

Min Heap Rule

In a min heap:

```
Parent <= Child
```

This means every parent must be less than or equal to its children.

So the smallest value is always at the root.

---

4. Beginner Intuition

Think of a heap like a tournament.

In a max heap:

- The winner stays at the top.
- Larger values stay above smaller values.
- The largest value is easy to find because it is at the root.

In a min heap:

- The smallest value stays at the top.
- Smaller values stay above larger values.
- The smallest value is easy to find because it is at the root.

Important idea:

> A heap does not fully sort all values. It only keeps the most important value at the root and maintains parent-child order.

---

5. Why Heaps Are Stored in Arrays

A heap is drawn like a tree, but in memory it is usually stored in an array.

Why?

Because a complete binary tree has no gaps. Since there are no gaps, we can store all nodes side by side in an array.

We do not need pointer fields like:

```
left child pointer
right child pointer
```

Instead, we can calculate parent and child positions using simple formulas.

---

6. Indexing Convention Used in This Curriculum

In this curriculum, the heap array starts at index `1`.

Index `0` is ignored.

Example:

```
Index:  0   1   2   3   4   5   6   7
Value:  -  40  35  30  15  10  25   5
```

The active heap starts from index `1`:

```
A[1] = 40
A[2] = 35
A[3] = 30
A[4] = 15
A[5] = 10
A[6] = 25
A[7] = 5
```

`A[0]` is ignored.

Why ignore index `0`?

Because it makes the formulas very simple.

---

7. Heap Array Representation

Array:

```
Index:  1   2   3   4   5   6   7
Value: 40  35  30  15  10  25   5
```

This represents the tree:

```
          40
        /
     35       30
   /   \     /
 15    10  25    5
```

The tree is logical. It is not physically stored using tree links.

The array positions decide the tree relationships.

For example:

- `A[1] = 40` is the root
- `A[2] = 35` is the left child of `40`
- `A[3] = 30` is the right child of `40`
- `A[4] = 15` is the left child of `35`
- `A[5] = 10` is the right child of `35`
- `A[6] = 25` is the left child of `30`
- `A[7] = 5` is the right child of `30`

---

8. Important Heap Formulas

For a node at index `i`:

Relationship	Formula
Left child	`2 * i`
Right child	`2 * i + 1`
Parent	`i / 2`

Integer division is used.

That means decimals are ignored.

Example:

```
5 / 2 = 2
7 / 2 = 3
```

---

9. Formula Example

Use this heap array:

```
Index:  1   2   3   4   5   6   7
Value: 40  35  30  15  10  25   5
```

Suppose we are at index `2`.

```
A[2] = 35
```

Find the left child

Formula:

```
Left child = 2 * i
```

So:

```
2 * 2 = 4
```

Therefore:

```
Left child is A[4] = 15
```

Find the right child

Formula:

```
Right child = 2 * i + 1
```

So:

```
2 * 2 + 1 = 5
```

Therefore:

```
Right child is A[5] = 10
```

Find the parent

Formula:

```
Parent = i / 2
```

So:

```
2 / 2 = 1
```

Therefore:

```
Parent is A[1] = 40
```

So index `2`, value `35`, has:

```
Parent = 40
Left child = 15
Right child = 10
```

---

10. Max Heap

A **max heap** keeps the largest value at the root.

Rule:

```
Parent >= Child
```

Example:

```
        40
      /
    35       30
   /  \     /
 15   10  25   5
```

Check the rule:

```
40 >= 35 and 40 >= 30
35 >= 15 and 35 >= 10
30 >= 25 and 30 >= 5
```

So this is a valid max heap.

The root is `40`, which is the largest value.

---

11. Important Point: A Heap Is Not Fully Sorted

A common beginner mistake is thinking that a heap is fully sorted.

It is not.

Example max heap array:

```
40, 35, 30, 15, 10, 25, 5
```

This is not sorted in descending order because `25` comes after `10`.

But it is still a valid heap.

Why?

Because the heap only cares about parent-child relationships.

It does not care whether siblings are sorted.

For example:

```
        40
      /
   35       30
```

`35` and `30` are siblings.

The heap does not require all siblings to be arranged in perfect order beyond the parent-child rule.

Main idea:

> A heap is partially ordered, not fully sorted.

---

12. Min Heap

A **min heap** keeps the smallest value at the root.

Rule:

```
Parent <= Child
```

Example:

```
        5
      /
    10      15
   / \     /
 20  25  30  40
```

Check the rule:

```
5 <= 10 and 5 <= 15
10 <= 20 and 10 <= 25
15 <= 30 and 15 <= 40
```

So this is a valid min heap.

The root is `5`, which is the smallest value.

---

13. Max Heap vs Min Heap

Feature	Max Heap	Min Heap
Root contains	Largest value	Smallest value
Parent-child rule	Parent >= child	Parent <= child
Main use	Quickly get maximum	Quickly get minimum
Example root	`40` in a max heap	`5` in a min heap

Simple memory trick:

```
Max heap = maximum at the top
Min heap = minimum at the top
```

---

14. Are Duplicates Allowed in a Heap?

Yes, duplicates are allowed.

Example max heap:

```
        40
      /
    40       30
   /
 20   20
```

This is valid because the max heap rule is:

```
Parent >= Child
```

Equal values are allowed.

So:

```
40 >= 40
20 >= 20
```

These are valid comparisons.

A heap does not require all values to be unique.

---

15. Height of a Heap

Because a heap is a complete binary tree, it stays compact and balanced.

It does not become a long chain like this:

```
40

   35

      30

         20
```

Instead, it stays compact like this:

```
         40
       /
    35        30
   /   \     /
 20    10   25    5
```

Because it stays compact, the height grows slowly.

For `N` elements:

```
Height = O(log N)
```

This is important because many heap operations move only up or down the height of the tree.

So later we will get:

```
Insertion = O(log N)
Deletion = O(log N)
```

---

16. Why Is Heap Height `O(log N)`?

Each level of a complete binary tree can hold more nodes than the previous level.

Example:

```
Level 1: 1 node
Level 2: 2 nodes
Level 3: 4 nodes
Level 4: 8 nodes
```

The number of nodes grows quickly as levels increase.

So even if the heap has many values, the number of levels is not very large.

That is why the height is logarithmic.

Simple meaning:

> A heap with many elements is still not very tall.

---

17. Why Heaps Are Not Good for Searching

A heap is good for quickly finding the root.

In a max heap:

```
A[1] = largest value
```

In a min heap:

```
A[1] = smallest value
```

But a heap is not good for finding any random value.

Example:

```
        40
      /
    35      30
   /  \    /
 15   10  25   5
```

If we ask:

```
Where is 25?
```

The heap does not give us a direct answer.

Why?

Because a heap is only partially ordered.

We know:

```
40 is above 35 and 30
35 is above 15 and 10
30 is above 25 and 5
```

But we do not know the exact location of every value from the heap rule alone.

So to search for `25`, we may have to scan many elements.

Therefore:

```
Heap is good for max/min access.
Heap is not good for general searching.
```

---

18. What Heaps Are Good For

Heaps are useful when we repeatedly need the most important value.

They are good for:

- Getting the maximum quickly
- Getting the minimum quickly
- Inserting efficiently
- Deleting the root efficiently
- Priority queues
- Heap sort

They are not mainly used for:

- Searching for a random value
- Keeping all values fully sorted at all times

---

19. Why Gaps Are Not Allowed

The complete binary tree rule is very important.

A heap must not have holes in the middle.

Example heap array:

```
Index:  1   2   3   4
Value: 30  20  15  10
```

Tree:

```
        30
       /
    20     15
   /
 10
```

This is complete.

---

20. Removing the Last Element Is Okay

If we remove the last value `10`, the array becomes:

```
Index:  1   2   3
Value: 30  20  15
```

Tree:

```
      30
     /
   20    15
```

This is still complete.

Why?

Because the blank space is only at the end.

There is no hole in the middle.

---

21. Removing a Middle Element Is Not Okay

If we remove the middle value `20`, the array becomes:

```
Index:  1   2   3   4
Value: 30  _   15  10
```

Now there is a hole at index `2`.

That breaks the heap.

Why?

Because index formulas become incorrect.

For example, index `4` should be the left child of index `2`:

```
Parent of index 4 = 4 / 2 = 2
```

But index `2` is empty.

So the tree structure no longer makes sense.

Main rule:

> Heap operations must preserve the compact left-to-right shape.

This is why:

- Insertion always happens at the next free position.
- Deletion removes the root, but replaces it using the last element.

---

22. Complete Binary Tree vs Incomplete Binary Tree

Feature	Complete Binary Tree	Incomplete Binary Tree
Array layout	No empty gaps	May contain holes
Shape	Compact and filled left to right	Missing nodes may appear in the middle
Formulas	`2i`, `2i + 1`, and `i / 2` work cleanly	Formulas may point to wrong positions
Suitable for heap?	Yes	No

---

23. C Program: Printing Heap Relationships

This program does not insert or delete values.

It only shows how the heap array formulas work.

```c
#include <stdio.h>

int main() {
    // Index 0 is ignored. Active heap elements are from index 1 to n.
    int A[] = {0, 40, 35, 30, 15, 10, 25, 5};
    int n = 7;

    printf("Index Value ParentIndex LeftChildIndex RightChildIndex\n");

    for (int i = 1; i <= n; i++) {
```

```
        int parent = i / 2;
        int left = 2 * i;
        int right = (2 * i) + 1;

        printf("%5d %5d ", i, A[i]);

        if (i == 1)
            printf("%11s ", "None");
        else
            printf("%11d ", parent);

        if (left <= n)
            printf("%14d ", left);
        else
            printf("%14s ", "None");

        if (right <= n)
            printf("%15d\n", right);
        else
            printf("%15s\n", "None");
    }

    return 0;
}
```

---

24. Explanation of the Program

The array is:

```
int A[] = {0, 40, 35, 30, 15, 10, 25, 5};
```

Here:

```
A[0] = 0, ignored
A[1] = 40
A[2] = 35
A[3] = 30
A[4] = 15
A[5] = 10
A[6] = 25
A[7] = 5
```

The variable `n` means there are 7 active heap elements.

```
int n = 7;
```

The loop runs from index `1` to index `7`:

```
for (int i = 1; i <= n; i++)
```

For each index, the program calculates:

```
int parent = i / 2;
int left = 2 * i;
int right = (2 * i) + 1;
```

Then it prints the current index, value, parent index, left child index, and right child index.

---

25. Dry Run for Index 3

Use the array:

```
Index:  1   2   3   4   5   6   7
Value: 40  35  30  15  10  25   5
```

For `i = 3`:

```
A[3] = 30
```

Parent

```
Parent index = 3 / 2 = 1
Parent value = A[1] = 40
```

Left child

```
Left child index = 2 * 3 = 6
Left child value = A[6] = 25
```

Right child

```
Right child index = 2 * 3 + 1 = 7
Right child value = A[7] = 5
```

So index `3` has:

```
Value = 30
Parent = 40
Left child = 25
Right child = 5
```

Check max heap rule:

```
30 >= 25
30 >= 5
```

So this local parent-child relationship is valid.

---

26. Expected Partial Output

The program will print something like:

```
Index Value ParentIndex LeftChildIndex RightChildIndex
    1    40        None              2               3
    2    35           1              4               5
    3    30           1              6               7
    4    15           2           None            None
```

Meaning:

- Index `1` is the root, so it has no parent.
- Index `1` has children at indexes `2` and `3`.
- Index `2` has parent index `1`.
- Index `2` has children at indexes `4` and `5`.
- Index `4` has no children because `8` and `9` are outside the heap size.

---

27. Complexity of the Program

The program visits every node once.

So the time complexity is:

```
O(N)
```

It only uses a few extra variables:

```
parent
left
right
i
```

So the extra space complexity is:

```
O(1)
```

---

28. Common Beginner Mistakes

Mistake	Why It Happens	How to Avoid It
Using index `0` as the root while using `2i`, `2i + 1`, `i / 2`	These formulas assume root starts at index `1`	In this curriculum, always ignore index `0`
Thinking a heap is fully sorted	The root is max or min, so learners think the whole structure is sorted	Remember: only parent-child order is guaranteed
Deleting from the middle	Arrays allow deletion from any index, but heaps need complete shape	Standard heap deletion removes the root
Forgetting duplicates are allowed	Some data structures do not allow duplicates	Heap rules allow equal values
Using a heap for normal searching	Root is easy to access, so learners think all values are easy to find	Use heaps for max/min access, not random search
Forgetting the complete tree rule	Learners focus only on values	Always check both shape and order

---

29. Phase 1 Key Terms

Term	Simple Meaning
Heap	A special binary tree with shape and order rules
Binary tree	A tree where each node has at most two children
Complete binary tree	A tree filled level by level from left to right
Root	The top node of the tree
Parent	A node above another node
Child	A node below another node
Max heap	Heap where parent is greater than or equal to children
Min heap	Heap where parent is less than or equal to children
Height	Number of levels from root to deepest leaf
Duplicate	Repeated value
Partially ordered	Some order exists, but the whole array is not sorted

---

30. Important Formulas to Memorize

Because this curriculum uses 1-based indexing:

```
Left child of index i  = 2 * i
Right child of index i = 2 * i + 1
Parent of index i      = i / 2
```

Example:

For index `5`:

```
Left child  = 2 * 5 = 10
Right child = 2 * 5 + 1 = 11
Parent      = 5 / 2 = 2
```

If the child index is greater than `n`, then that child does not exist.

---

31. How to Check If an Array Is a Max Heap

Suppose we have this array:

```
Index:  1   2   3   4   5   6   7
Value: 50  30  40  10  20  35  25
```

To check if it is a max heap, check every parent with its children.

Index 1

```
A[1] = 50
Children: A[2] = 30, A[3] = 40
50 >= 30 and 50 >= 40
```

Valid.

Index 2

```
A[2] = 30
Children: A[4] = 10, A[5] = 20
30 >= 10 and 30 >= 20
```

Valid.

Index 3

```
A[3] = 40
Children: A[6] = 35, A[7] = 25
40 >= 35 and 40 >= 25
```

Valid.

Therefore, this is a valid max heap.

---

32. How to Check If an Array Is a Min Heap

Suppose we have this array:

```
Index:  1   2   3   4   5   6   7
Value: 5   10  15  20  25  30  40
```

To check if it is a min heap, check every parent with its children.

Index 1

```
A[1] = 5
Children: A[2] = 10, A[3] = 15
5 <= 10 and 5 <= 15
```

Valid.

Index 2

```
A[2] = 10
Children: A[4] = 20, A[5] = 25
10 <= 20 and 10 <= 25
```

Valid.

Index 3

```
A[3] = 15
Children: A[6] = 30, A[7] = 40
15 <= 30 and 15 <= 40
```

Valid.

Therefore, this is a valid min heap.

---

33. Small Practice Questions

Use this heap:

```
Index:  1   2   3   4   5   6   7
Value: 50  30  40  10  20  35  25
```

Tree:

```
        50
      /
    30        40
   /   \     /
 10    20   35   25
```

Questions

1. What is the root?
2. What are the children of index `2`?
3. What is the parent of index `6`?
4. Is this a max heap?
5. What is the left child index of index `3`?
6. What is the right child index of index `3`?
7. Is the array fully sorted?

---

34. Practice Answers

1. What is the root?

The root is at index `1`.

```
A[1] = 50
```

Answer:

```
50
```

2. What are the children of index `2`?

Use the formulas:

```
Left child = 2 * i
Right child = 2 * i + 1
```

For `i = 2`:

```
Left child = 2 * 2 = 4
Right child = 2 * 2 + 1 = 5
```

So:

```
A[4] = 10
A[5] = 20
```

Answer:

```
10 and 20
```

3. What is the parent of index `6`?

Use the formula:

```
Parent = i / 2
```

For `i = 6`:

```
6 / 2 = 3
```

So:

```
A[3] = 40
```

Answer:

```
40
```

4. Is this a max heap?

Check parent-child relationships:

```
50 >= 30 and 50 >= 40
30 >= 10 and 30 >= 20
40 >= 35 and 40 >= 25
```

All are true.

Answer:

```
Yes, it is a max heap.
```

5. What is the left child index of index `3`?

```
Left child = 2 * 3 = 6
```

Answer:

```
Index 6
```

6. What is the right child index of index `3`?

```
Right child = 2 * 3 + 1 = 7
```

Answer:

```
Index 7
```

7. Is the array fully sorted?

Array:

```
50, 30, 40, 10, 20, 35, 25
```

This is not fully sorted.

But it is still a valid max heap.

Answer:

```
No. A heap is not fully sorted.
```

---

35. Final Phase 1 Summary

A heap is not just any tree.

A heap is:

```
Complete binary tree + heap ordering rule
```

The complete binary tree rule means:

```
No gaps. Fill from left to right.
```

The heap ordering rule means:

```
Max heap: parent >= child
Min heap: parent <= child
```

The heap is usually stored in an array.

In this curriculum:

```
Index 0 is ignored.
Root starts at index 1.
```

Important formulas:

```
Left child  = 2i
Right child = 2i + 1
Parent      = i / 2
```

A heap is good for:

```
Getting maximum quickly
Getting minimum quickly
Priority queues
Heap sort
```

A heap is not good for:

```
Searching for random values
Keeping the whole array fully sorted at all times
```

Most important beginner idea:

> A heap is a compact tree stored inside an array, where the most important value is always at the root.

---

36. What You Must Know Before Phase 2

Before moving to Phase 2, make sure you can answer these:

1. What is a heap?
2. What is a complete binary tree?
3. Why does a heap not allow gaps?
4. Why is index `0` ignored in this curriculum?
5. What is the formula for the left child?
6. What is the formula for the right child?
7. What is the formula for the parent?
8. What is a max heap?
9. What is a min heap?
10. Why is a heap not fully sorted?
11. Why is heap height `O(log N)`?
12. Why is a heap not good for searching random values?

If you understand these, you are ready for Phase 2: insertion and in-place heap creation.